

Workshop 12

Name: Kaung Khant Lin — **ID:** 6540131

Course/Section: CSX3004 / 542

Task 1 — Cost for iterating over a list of size n

The loop body performs a single addition per element (e.g., `total += x`). Executed n times, the total operation count scales linearly with n .

Asymptotic cost: $\Theta(n)$.

Task 2 — Recurrence for `factorial(n)` under the given cost model

Let $C(n)$ count (**multiplications + recursive-call overhead**).

- **Base:** $C(0) = 0$ (immediate return).
- **Step:** for $n > 0$, do one multiplication and one recursive call:

$$C(n) = \{0 \text{ if } n = 0, 2 + C(n - 1) \text{ if } n > 0\}$$

Thus, the work grows in direct proportion to n : $\Theta(n)$.

Task 3 — Reversing a list: append-based vs. accumulator

Cost definition here: number of `append/3` calls only.

Version 1 (append at each step)

```
rev([H|T], R) :- rev(T, RT), append(RT, [H], R).
rev([], []).
```

- If $C1(n)$ is the number of `append/3` calls on a list of length n :

$$C1(0) = 0, C1(n) = 1 + C1(n - 1) \Rightarrow C1(n) = n.$$

Version 2 (accumulator, no append)

```
rev(L, R)                :- rev_helper(L, [], R).  
rev_helper([], A, A).  
rev_helper([H|T], A, R) :- rev_helper(T, [H|A], R).
```

- Uses no **append/3**: $C_2(n)=0$ under this metric.

Which is better under this metric? Version B, because it eliminates **append/3**.

Task 4 — If the model counts function/predicate calls instead

The cost model is changed to count function calls.

1) How does the cost of factorial change?

The factorial(n) function calls itself n times (factorial($n-1$), factorial($n-2$), ..., factorial(0)) plus the initial call.

- The total number of calls for factorial(n) is $n + 1$.
- This is still a linear cost, $O(n)$. The original cost $C(n) = 2n$ and the new cost $C'(n) = n+1$ have the same linear complexity.

2) How does the cost of list reversal change?

- Version 1: This version calls rev recursively n times and append n times. Including the initial call to rev, the total number of function calls is $(n+1) + n = 2n + 1$.
- Version 2: The main rev predicate makes one call to rev_helper. The rev_helper predicate then calls itself n times. Including the initial rev call and the base case call for rev_helper, the total is $1 + (n+1) = n + 2$.

Even with a linear cost of $O(n)$ under this new model, Version 2 maintains greater efficiency due to approximately half the number of function calls. Additionally, this cost model obscures the fact that Version 1's **append** calls are significantly more computationally intensive than the straightforward list construction in Version 2.

Task 5 — Row-major vs column-major access patterns

1) In C, which loop order is faster, and why?

Row-major inner loop is faster. C arranges 2-D arrays in row-major order; scanning along the row touches contiguous memory, yielding many cache hits per cache line. Column-wise traversal jumps by a full row stride, causing frequent cache misses. **Conclusion:** row-major inner loop wins due to spatial locality.

2) Timings:

- Row-major time: **0.238337 sec**
- Column-major time: **0.446370 sec**

3) If written in Fortran/MATLAB

Those use column-major storage; therefore the **column-major** inner loop becomes the cache-friendly one and should be faster.

4) If using `malloc` in C,

- **Single contiguous block** (`int* a = malloc(N*N*sizeof(int));`) preserves row-major contiguity—same result as static arrays.
- **Array of row pointers** (each row contiguous) still favors row-major scans for good locality within each row. Either way, the conclusion about loop order doesn't change.

5) Python & NumPy Test:

- Row-major access time (via Python loops): 15.982165 sec
- Column-major access time (via Python loops): 17.460288 sec
- NumPy's optimized `np.sum()` time: 0.020013 sec